REPOSITORY OVERVIEW

FoodLoop

Connecting surplus food with the people who need it

Next.js · Express · MySQL · MongoDB



One platform, end to end



Post

Restaurants, hotels and bakeries list surplus food with quantity, diet, storage and a pickup deadline.



Match

An admin approves each listing, then it is auto-matched to the closest open NGO requirement.



Deliver

A volunteer picks up and delivers, and the NGO rates the donor to close the loop.

4
user roles

7
workflow stages

2
databases

Four roles, one workflow



Donor

Restaurants, hotels and bakeries post surplus food with a pickup deadline.



Acceptor (NGO)

NGOs, shelters and children's homes post needs and claim matched food.



Volunteer

Picks up and delivers food, and can switch availability on or off.



Admin

Approves or rejects listings, runs matching and assigns volunteers.

The donation lifecycle



Posted
Donor lists surplus

Approved
Admin reviews listing

Claimed
NGO accepts match

Assigned
Volunteer allocated

Picked Up
Food leaves donor

Delivered
Reaches the NGO

Completed
NGO leaves a rating

An admin can reject a posted listing with a written reason (up to 200 characters).

The API only allows the next step in order. Anything else returns HTTP 409.

Smart matching

- Listing has not passed its deadline
- Requirement is open and not declined before
- Diet fits the NGO's preference
- Category matches, or the NGO accepts any
- Donation covers at least half the meals needed
- Within 15 km, or 25 km if the NGO has transport

THEN, THE BEST FIT WINS

distance (km)
+ meal gap ÷ 20

meal gap = |donated − needed|

Lowest score is picked. Road distance is estimated as 1.3× the straight line.

Urgency at a glance

Critical

< 2 h

Needs action right away

Urgent

< 6 h

Pick up soon

On track

6 h +

Plenty of time left

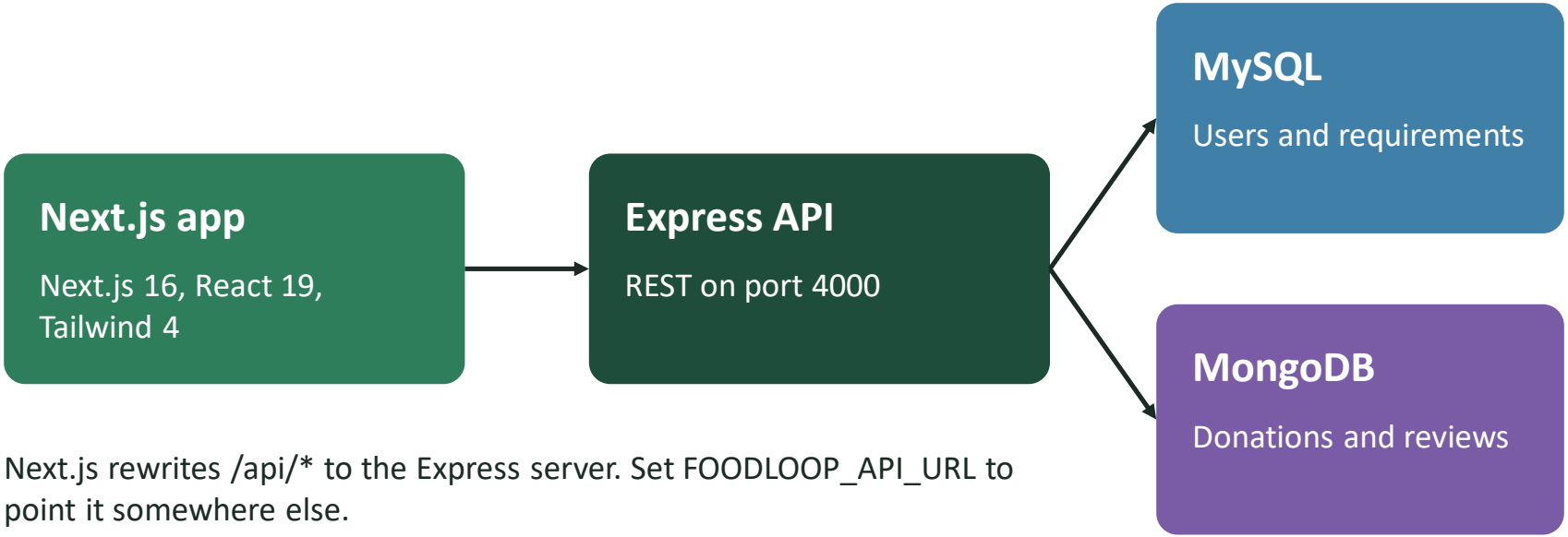
Expired

Past

Deadline gone, no matching

The app clock ticks every 30 seconds, so urgency badges and time-left labels stay current without a refresh.

Architecture at a glance



Next.js rewrites /api/* to the Express server. Set FOODLOOP_API_URL to point it somewhere else.

Two databases



MySQL

Relational data

- Users: donors, NGOs, volunteers, admin
- Requirements, with categories stored as JSON



MongoDB

Document data

- Donations, including status history
- Reviews with ratings and comments

On first start the API seeds sample users and requirements into MySQL, and sample donations and reviews into an empty MongoDB collection. Existing records are kept.

REST API

Endpoint	What it does
`GET /api/health`	Health check
`GET /api/donations`	List donations, filter by status, category or donor
`GET /api/donations/:id`	Get one donation
`POST /api/donations`	Create a validated listing
`PATCH /api/donations/:id`	Edit a listing
`PATCH /api/donations/:id/status`	Advance the workflow, or reject
`DELETE /api/donations/:id`	Remove a listing
`GET /api/requirements`	List NGO requirements
`GET /api/users`	List users, optionally by role
`POST /api/reviews`	Rate a delivered donation (1 to 5)

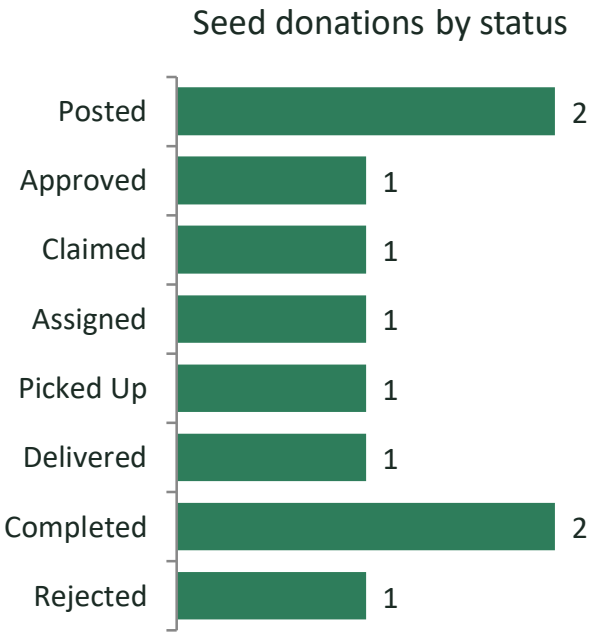
Sample data included

10
users across all four roles

10
donations at every stage

8
NGO requirements

10
Bengaluru neighborhoods



Run it locally

1. Start a MySQL and a MongoDB server
2. Add the settings to `.env.local`
3. `pnpm install`
4. `pnpm api` API on localhost:4000
5. `pnpm dev` web app, proxies /api/*

SUPPORTED SETTINGS

```
PORT
MYSQL_HOST
MYSQL_PORT
MYSQL_USER
MYSQL_PASSWORD
MYSQL_DATABASE
MONGODB_URI
MONGODB_DATABASE
```



From surplus to supper

- Four roles, one shared workflow
- Rule-based matching with urgency tracking
- Express API backed by MySQL and MongoDB